

# Cost-Aware Repository Change Localization with Sparse Impact

## Planning and Bounded Verification

MSc Research Proposal (V1.4, supervisor-ready / print-candidate)

Ahmed Ehab

2026-09-17

### Abstract

Large language models (LLMs) that edit a software repository must first decide which files a requested change affects. This decision is costly: an explicit file-by-file impact policy over a repository-scale candidate universe spends hundreds of serialized records per call, and a second-stage reasoner that revisits the scope can multiply inference cost by orders of magnitude.

*Central identity of the thesis:*

This thesis studies whether a cheap sparse file-impact policy can reduce repository-wide localization effort while a bounded second look recovers important files the first pass chose not to change.

We investigate a representation that is cheap by construction — *preserve-by-omission* sparse impact plans — and evaluate it with cheap lexical/static baselines and an explicit *bounded* second-stage omission-recovery mechanism, all under matched inference budgets. Preliminary development evidence shows a large serialization-cost reduction for the sparse representation, a meaningful zero-LLM lexical signal, and — for the first time at candidate level — a robust recovery signal: a frozen BM25+Graph-Neighbor composite ranker (historical label: Classical-CIA) recovers Sparse-omitted missed files above the analytic-Random baseline across the whole budget curve, with task-level bootstrap intervals excluding zero. A clean second-repository DEVELOPMENT run (Saleor, 150 tasks) transfers positively: the frozen composite beats analytic Random on 149 evaluable tasks (B=5 delta +0.231, CI [+0.180,+0.287]), a DEVELOPMENT transfer replication — not a confirmatory result. An identity audit and incremental-evidence ablation show the signal is predominantly lexical (BM25), with the binary graph-neighbor term contributing a small, largely non-significant increment; the thesis therefore does not claim graph novelty. Task-level omission risk is not reliably predictable on current development evidence, so the thesis does not depend on a task-level risk scorer.

## 1 Background, motivation, and problem

Repository-level coding agents and impact planners must select affected files before editing. Two problems motivate the thesis. First, *impact inference*: did the model identify the right files? Second, *impact-policy representation*: can a complete file-action policy be expressed without spending output budget on hundreds of repeated *preserve* decisions? Our controlled study shows that an explicit full policy at a 16K completion cap serializes on average 144 decision records per candidate universe, while a sparse policy serializes on average 4.9 records (a ~96.6% reduction), with large associated reductions in completion tokens and cost.

Second-stage reasoning is expensive: a graph-guided agent evaluated under a shared protocol consumed on the order of a million tokens per task on the same held-out tasks where our sparse first pass stays below a few thousand tokens. If a cheap first pass already localizes the bulk of

affected files, the question is whether a *bounded* second look can recover missed files without paying the full always-on cost.

Given a repository revision  $P$  (parent commit), a change request  $q$ , and a candidate universe  $U(P)$  visible at  $P$ , a localization system produces an affected-file set  $\hat{S}(P, q)$ ; the hidden observed change set  $S^*(P)$  is used for evaluation only. The observed historical diff is an *observed change-set proxy*, never semantic impact gold (Section 6).

## 2 Central thesis hypothesis and gap

*Hypothesis.* A sparse/cheap first-pass localization policy followed by a bounded omission-recovery mechanism can dominate both cheap-localization-only and always-on expensive reasoning on the correctness–cost Pareto frontier, for the repositories and protocols we evaluate.

Everything else supports this story:

- *Preserve-by-Omission* = representation / first-pass contribution;
- *BM25 / classical static change-impact analysis* = cheap baselines;
- *graph-guided agent* = expensive system-level context;
- *task-level omission-risk scorer* = investigated and currently rejected on development evidence;
- *candidate-level bounded verification* = primary next mechanism;
- *second and third repositories* = generalization.

Task-level RiskScorer success is *not* a thesis requirement. Existing work covers several individual components; this thesis evaluates *their combination* as a sparse first pass plus bounded omission recovery under matched budgets (Section 4).

## 3 Research questions and expected contributions

**RQ1.** How does sparse explicit impact-policy representation affect inference/output cost and file-selection fidelity?

**RQ2.** How accurately can cheap lexical/static methods localize impacted files relative to LLM planners under shared protocols?

**RQ3.** Can bounded second-stage verification recover missed impacted files more efficiently than always-on or random matched-budget verification?

**RQ4.** How do these correctness–cost trade-offs transfer across repositories and ecosystems?

If task-level risk modeling remains unsupported, RQ3 is answered by the candidate-level mechanism. *Expected contributions:* (1) sparse explicit impact-plan representation with controlled evidence; (2) a reproducible real-commit impact-localization dataset and protocol; (3) fair cheap/classical/LLM/agent comparison under common file-level metrics and explicit budgets; (4) a bounded/selective-compute verification protocol (candidate-level by default); (5) cross-repository correctness–cost evidence; (6) reproducibility artifacts and leakage-control methodology.

No positive algorithmic result is promised.

## 4 Positioning against related work

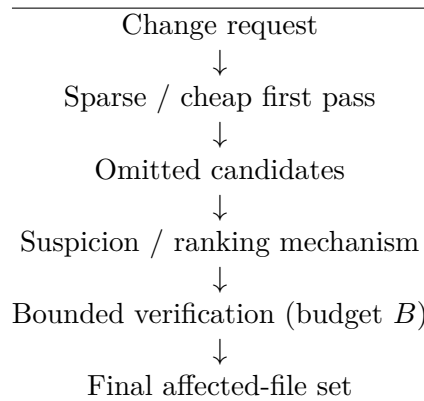
Existing work covers several individual components. This thesis evaluates their combination. Verified related lines (primary sources, 2026-09-16/17):

- *Classical change-impact / ripple effect (RIPPLE line)*. RIPPLE = a known seed  $\rightarrow$  recall expansion  $\rightarrow$  precision refinement. This thesis differs: an explicit repository-wide sparse policy  $\rightarrow$  a rejected residual set  $\rightarrow$  bounded reconsideration of rejected candidates.
- *Change-Patterns Mapping* (TSE 2022): boosts/reranks the initial impact set from historical patterns. This thesis allocates *new inspection to the excluded residual set* rather than reranking the selected set.
- *ArtifactSync*: a demo-level mechanistic analogue (uncertainty- driven deeper context) but a different post-commit synchronization task, not file-impact localization.
- *Modern transform + program-dependence-graph impact analysis* (Yan et al. 2026): graph+LLM impact analysis is active; our graph is one optional verifier.
- *Selective-compute / efficiency+verification priors* (Repoformer, FastCoder): selective retrieval / efficient verification at repository level for code-completion/generation tasks, guarding generic “selective escalation is novel” claims.
- *Graph-guided issue localization* (LocAgent, GraphLocator, LLM-driven iterative code-graph search): expensive graph-guided competitors.

Graph, history, and selective computation are *not* novelty claims by themselves. The comparison matrix (Section 7) records whether each line has an explicit file policy, sparse omission representation, cheap first pass, omission recovery, hard/matched budget, real-commit evaluation, and cross-repository evidence. Where novelty is only defensible once experiments support it, it is marked *candidate novelty — not yet claimed*.

## 5 Proposed two-stage architecture

1. **Sparse/cheap first-pass impact policy**: explicit file-level policy with preserve-by-omission serialization.
2. **Candidate-level bounded verification** (primary): rank the first-pass omitted candidates with independent structural / retrieval / history evidence, and verify only those under a hard per-task budget  $B$ . Compare with always-verify and random matched-budget verification.



## 6 Dataset construction, labels, and leakage controls

- **Real history, parent-only inputs.** Inference uses the parent commit  $P$ ; the target  $T$  is hidden. Live HEAD is never used.
- **Eligibility and exclusions.** Single-parent commits with a meaningful intent, touching only production source files; tests, configurations, docs, migrations, and generated files are excluded; intent-path leakage is a hard exclusion.
- **Deduplication.** Exact-proxy-set / shared-reference / suspected- related changes are removed with frozen deterministic rules.
- **Observed change-set proxy.**  $P \rightarrow T$  changed production files are an *observed* proxy, never semantic impact gold. Localization claims are scoped to historical-file recovery unless semantic adjudication supports more.
- **Splits.** Metadata-only, deterministic split frozen before any model result. For djangoCMS V2-LARGE: development, an untouched internal test, and a reserve. The v1 40 are LEGACY-EXPOSED and never reused as untouched confirmation.

## 7 Experimental design, baselines, budgets, metrics, statistics

### 7.1 Budget curve, not a single point

The primary scientific object is the *budget curve*  $B \in \{0, 1, 3, 5, 10\}$ ;  $B=0$  is Sparse alone.  $B=5$  is a convenient reference operating point, not a scientifically privileged single claim. We also report the normalized sensitivity  $B/|\text{omitted set}|$  as a secondary view.

### 7.2 Analytic Random control

For ranking-only experiments the Random control is the analytic hypergeometric expectation: for a task with  $N_t$  omitted candidates,  $M_t$  Sparse-observed FNs, and budget  $B$ ,  $\mathbb{E}[X_t] = B \cdot M_t / N_t$  (with  $B$  clipped to  $N_t$ ). The comparison is per-task observed recovery vs this analytic expectation, aggregated with task-level bootstrap intervals. Random repetitions are never independent tasks.

### 7.3 Primary endpoint

*False-Negative Recovery Rate / Omission Recovery Rate @ B*: per task, recovered Sparse-observed FNs within  $B$  / all Sparse-observed FNs. Secondary: absolute recall gain, FNR reduction, fraction of Oracle@ $B$  gap closed, candidates inspected per recovered FN, precision/F1 (safety metrics), runtime, index/build cost.

### 7.4 Anchors (per B)

Sparse/ $B=0$ ; Analytic Random; BM25; Path-token; Graph/static; BM25+Graph-Neighbor Composite (historical label: Classical-CIA; the frozen primary); history/co-change (where parent-visible history is available); one fixed simple hybrid (0.5 BM25 + 0.5 graph neighbor, a rank-equivalent redundant control — confirmed by the ranker-identity audit, not an independent baseline); Oracle (ranking headroom); InspectAll (exhaustive). *Oracle@B*  $\neq$  *InspectAll*: Oracle is ranking headroom; InspectAll is exhaustive reconsideration.

| Study            | Purpose             | Repo              | Split      | Methods          | Unit        | Reps | Budget            | Metrics                       | Status          |
|------------------|---------------------|-------------------|------------|------------------|-------------|------|-------------------|-------------------------------|-----------------|
| Representation   | parse-vs-full cost  | djangoCMS curated | controlled | Full Sparse      | vs task     | 3    | cap 16K           | tokens, records, cost, P/R/F1 | done            |
| Real-commit      | selection fidelity  | djangoCMS         | held-out   | Full Sparse      | vs task     | 3    | cap 16K           | P/R/F1/FNR                    | done            |
| Graph agent      | expensive context   | djangoCMS         | held-out   | agent            | task        | 1    | per-call          | P/R/F1, cost                  | done            |
| Cheap baselines  | zero-LLM signal     | djangoCMS         | lev        | BM25/graph       | task        | –    | cheap             | P/R/F1                        | done            |
| Task risk        | risk predictability | djangoCMS         | lev        | features         | task        | 3    | –                 | AUROC/AUPRC                   | done (neg.)     |
| Candidate verify | omission recovery   | djangoCMS         | lev        | rankers verifier | + candidate | –    | $B \in \{0..10\}$ | FN-recovery                   | dev + pilot     |
| Cross-repo       | transfer            | Saleor            | dev        | frozen method    | task        | –    | frozen            | per-repo                      | dev replication |

Table 1: Experimental Design at a Glance (internal IDs appear in traceability notes only).

## 7.5 Experimental Design at a Glance

## 7.6 Statistics and multiple-comparison discipline

Task is the independent unit; candidate rows are nested. One primary endpoint (recovery@ $B$ ); one primary arm-vs-Random comparison; secondary endpoints labeled; task-level paired bootstrap; no post-hoc feature-family winner; exploratory analyses reported as exploratory.

## 8 Preliminary evidence

All numbers are from audited studies in this repository; they are development or controlled evidence with their bounds.

- **Representation effect (controlled).** At a 16K completion cap, Sparse serialized on average **4.9** decision records vs **144.0** for Full (**~96.6%** reduction), with large completion-token and cost reductions. No universal semantic-superiority claim.
- **Historical real-commit study.** 60 cells over 10 held-out real djangoCMS commits: selection is difficult, FNR is high, and there is no Full-vs-Sparse superiority claim (paired deltas’ intervals cross zero). Sparse is substantially cheaper.
- **Graph-guided agent comparison.** System-level shared-protocol context: 402 calls / 32.8M tokens /  $\sim$ \$9.93 estimated on 10 tasks, with 5/10 non-usable outcomes (2 timeout + 1 context-length + 2 completed-but-empty). Not a reproduction of the published fine-tuned result.
- **Cheap/static development baselines.** BM25 provides a meaningful zero-LLM signal; graph-ranked seeds are comparable; a classical CIA (dependency propagation) baseline is implemented on 150 development cases.
- **Task-level omission-risk investigation.** Live Sparse development inference (90 cells; then 431 calls with a fail-closed stop at the 2.5M-token ceiling). A task-level RiskScorer is not justified: the v1 signal does not replicate within the new development cohort and the pooled signal is a candidate-universe-size artifact.
- **Candidate-level bounded verification (Route B, development).** A frozen BM25+Graph-Neighbor composite ranker (historical label: Classical-CIA; exact formula = normalized BM25 + binary graph-neighbor, per the ranker-identity audit) recovers

Sparse-omitted missed files above the analytic-Random baseline across the whole budget curve  $B \in \{1, 3, 5, 10\}$ , with task-level bootstrap intervals excluding zero at every point (e.g. at  $B=5$ : pooled macro recovery 0.163 vs random 0.029, interval  $[+0.09, +0.18]$ ), 5/5 development folds positive, and no omitted/universe-size artifact. A small verifier pilot (30 calls,  $\sim \$0.002$ ) found the frozen ranker places the recoverable missed files within the top- $B$  (Oracle-in-top- $B \approx 1.0$ ) and the verifier recovers  $\sim 0.86$ – $1.0$  of them, so the dominant loss is first-pass omission.

- **Saleor DEVELOPMENT transfer replication (2026-09-17).** A clean Saleor DEVELOPMENT Sparse run (150 tasks  $\times$  3 reps = 450 cells; 446 valid / 4 failed; 7,316,986 tokens /  $\sim \$2.31$ ; 0 truncations; no Saleor-specific tuning) enabled a frozen Route-B transfer test on 149 evaluable tasks: the composite beats analytic Random at every  $B$  ( $B=5$ : 0.237 vs 0.006, delta +0.231, CI  $[+0.180, +0.287]$ ; 5/5 folds positive; no size artifact). This is a **DEVELOPMENT transfer replication, NOT a confirmatory result**; djangoCMS INTERNAL\_TEST/RESERVE remain sealed.
- **Incremental-evidence ablation (zero API, 2026-09-17).** Composite-minus-BM25 paired task-level deltas are small with bootstrap CIs including zero at most  $B$  on both repositories (djangoCMS  $B=5$  +0.003  $[-0.015, +0.019]$ ; Saleor  $B=5$  +0.003  $[-0.017, +0.024]$ ); the replicated cross-repo signal is **predominantly lexical (BM25)**. The thesis does not claim graph novelty.
- **Classical/history evidence.** A parent-visible history/co-change arm also beats analytic Random where history is available; the BM25+Graph-Neighbor composite remains the frozen primary.

These results motivate a larger development dataset, protection of an untouched internal test, and the bounded-verification design; they are not rewritten as a success.

## 9 Cross-repository generalization

Stage 1 djangoCMS V2-LARGE (within-repo large evidence + untouched internal test)  $\rightarrow$  Stage 2 Saleor (different real system, same Python/Django stack)  $\rightarrow$  Stage 3 NestJS (cross-language / cross-framework evidence). Each stage states its claim boundary. NestJS is preferred but not forced; a predeclared eligible-pool yield criterion and a backup protocol are frozen.

### 9.1 Saleor DEVELOPMENT transfer replication (completed, 2026-09-17)

The Saleor DEVELOPMENT sparse run (150 $\times$ 3 = 450 cells; 446 valid / 4 failed; 7,316,986 tokens /  $\sim \$2.31$ ; 0 truncations; no Saleor-specific tuning) and the frozen Route-B transfer test (149 evaluable tasks) replicate the bounded omission-recovery signal on a second, much larger repository ( $B=5$  composite 0.237 vs analytic Random 0.006; delta +0.231, CI  $[+0.180, +0.287]$ ; 5/5 folds positive; no size artifact). This is **DEVELOPMENT transfer replication only — NOT confirmatory**. Saleor INTERNAL\_TEST/RESERVE are sealed and were never opened. Per-repository primary; no djangoCMS+Saleor pooling as the headline.

### 9.2 Scope contract

*CORE MSc*: Preserve-by-Omission representation evidence; fixed-budget Route-B development + a frozen confirmatory test; semantic-proxy audit; djangoCMS; Saleor as secondary replication if completed. *Conditional / Future*: adaptive  $B_t$ ; learned budget allocation; NestJS cross-language; end-to-end regeneration correctness; persistent memory; richer causal-graph verifier.

## 10 Threats, contingency, and timeline

### 10.1 Threats to validity

Internal: the observed proxy is not semantic gold; leakage control is mandatory; no hidden-test peeking. External:  $n$  and single-repository generalization; V2 and cross-repository evidence are the mitigation, with per-repository primary metrics. Statistical: low negatives, multiple comparisons, wide random bands — addressed by pre-registration, class-balance gates, analytic-Random controls, and bootstrap uncertainty. Representation: LLM serialization cost spans providers; both recorded and frozen-pricing cost are reported.

### 10.2 Semantic-proxy validity

Because the historical diff is only a proxy, we prepare a stratified development-only sample with evidence packets for later human adjudication (protocol: `docs/SEMANTIC_PROXY_AUDIT_PROTOCOL.md`). Machine-assisted notes are never semantic gold.

### 10.3 Contingency

If candidate-level bounded verification does not beat analytic Random at the frozen budget, the negative result is frozen and reported; no complex verifier is invented.

### 10.4 Future work: adaptive / cost-sensitive verification allocation

Inspired by the Shichao Zhang adaptive-resource line (Learning- $k$ , one-step KNN, cost-sensitive KNN, adaptive-neighborhood) and selective-prediction / learning-to-defer literature, we sketch a future adaptive budget:  $B_t^* = \arg \min_B [C_{\text{verify}}(B) + \rho \mathbb{E}[\text{remaining omission loss} \mid \text{evidence}, B]]$ , with  $\rho$  a sensitivity, not a claimed monetary truth. This is NOT yet a contribution and is NOT implemented before fixed-budget Route B is established.

### 10.5 Timeline

| Window     | Activity                                                                            |
|------------|-------------------------------------------------------------------------------------|
| 2026-09/10 | Proposal V1.3; V2-LARGE freeze + build                                              |
| 2026-10    | V2 development inference; cheap + classical baselines; candidate-level verification |
| 2026-11    | Bounded-verification experiments; Saleor protocol + inference                       |
| 2026-12    | NestJS cross-ecosystem check; Pareto analysis                                       |
| 2027-01/02 | Analysis, replication, threat revisions                                             |
| 2027-03/04 | Write-up; internal-test confirmation                                                |
| 2027-05    | Submission-ready thesis                                                             |

## 11 References

The authoritative bibliography is `references.bib` (V1.3). References were verified against primary sources (arXiv API / Crossref) on 2026-09-16/17. Preprints are marked. No fabricated or placeholder citations remain.